


# RL Project Report

Reinforcement Learning

TurtleBot3 Obstacle Avoidance using ROS 2

**Algorithms Implemented:**
Policy Gradient, Deep Q-Networks (DQN), and Actor-Critic

**Names:**
BOUGATEF Jasser
BEN SLIMENE Farah
SOULEIMANI HOUTI HASSANI Chahd

**Specialty:** IMR

**Date:** November 2, 2026

# Contents

**1 Introduction** **2**

**2 Environment Architecture and Setup** **2**
2.1 Task and Robot Environments . . . 2
2.2 State Observation and Sensor Modality . . . 2
2.3 Action Space . . . 3
2.4 Reward Strategy for Navigation . . . 3
2.5 Training Duration and Stopping Strategy . . . 3

**3 Algorithm Development and Training Debugging** **4**
3.1 Baseline: Rule-Based Obstacle Avoidance . . . 4
3.2 Algorithm 1: Policy Gradient (PG) . . . 4
3.2.1 Training Debugging and Fixes . . . 5
3.3 Algorithm 2: Deep Q-Network (DQN) . . . 6
3.3.1 Debugging, Hyperparameter Tuning, and Reward Redesign . . . 7
3.3.2 DQN Performance Analysis . . . 8
3.4 Algorithm 3: Actor-Critic (AC) . . . 8
3.5 Training Hyperparameters . . . 9

**4 Results and Algorithm Comparison** **10**
4.1 Discussion of Algorithm Performance . . . 10

**5 DQN with Curriculum Learning** **11**
5.1 Workspace and Node Architecture . . . 11
5.2 State Representation . . . 11
5.3 Action Space . . . 12
5.4 Reward Function . . . 12
5.5 Network Architecture and Training . . . 13
5.6 Curriculum Learning Strategy . . . 14
5.7 Evaluation Results . . . 14

**6 Conclusion** **16**

**Abstract**

This project explores the application of Deep Reinforcement Learning (DRL) for autonomous robotic navigation and obstacle avoidance. Using ROS 2 Humble and Gazebo Fortress, a TurtleBot3 was trained in a simulated environment using continuous LiDAR sensor data. Three distinct DRL algorithms—Policy Gradient, Deep Q-Networks (DQN), and Actor-Critic—were implemented from scratch in PyTorch, trained, and evaluated. The comparative analysis reveals that the Actor-Critic architecture provided the most stable and highest-performing policy for this continuous-state, discrete-action navigation task.

# 1 Introduction

This report details the implementation of a reinforcement learning (RL) framework to teach a TurtleBot3 robot to navigate and avoid obstacles. The architecture utilizes ROS 2 Humble, Gazebo Fortress, and Ubuntu 22.04. By leveraging the OpenAI Gym standard, an interface was built to bridge the simulation with deep learning scripts written in PyTorch. Three algorithms were developed and compared: Policy Gradient, Deep Q-Network (DQN), and an Actor-Critic method.

# 2 Environment Architecture and Setup

## 2.1 Task and Robot Environments

Within the project hierarchy (specifically inside `turtlebot_env.py`), the simulation relies on specialized environment classes that inherit from the `gym.Env` standard. This structure bridges the ROS 2 ecosystem with standard deep learning libraries:

- **Robot Environment:** This class specifies the robot used for the task. It manages all ROS 2 functionalities required to control the TurtleBot3, ensuring that critical topics (such as `/cmd_vel` for movement and `/scan` for perception) and services (such as `/reset_simulation`) are fully operational.
- **Task Environment:** This class dictates the core rules of the task the agent must learn. It defines how the chosen action is applied to the robot (`step(action)`), how observations are processed (`_get_obs()`), how rewards are computed (`_compute_reward()`), and evaluates the termination conditions (`_is_done()`).

## 2.2 State Observation and Sensor Modality

To observe the state of the environment, the TurtleBot3 relies on its 360-degree LiDAR sensor. The raw laser scan provides continuous distance readings in all directions. To prevent the neural network from being overwhelmed by high-dimensional data, this raw input is systematically downsampled.

A single state observation is defined as an array of **24 discrete laser distance readings**, which provides a sufficiently detailed yet computationally efficient representation of the robot's immediate surroundings. Specifically, in `_get_obs()`, every 15th reading of the 360-degree scan is sampled. Infinite values (representing distances out of the sensor's range) are replaced with a maximum range of 3.5 meters, and all observations are clipped strictly to this [0.0, 3.5] valid range.

### 2.3 Action Space

The RL algorithms predict from a discrete action space consisting of **3 distinct actions**, which are translated into `Twist` commands:

1. **Move Forward:** (`cmd.linear.x = 0.2`, `cmd.angular.z = 0.0`)
2. **Turn Left:** (`cmd.linear.x = 0.1`, `cmd.angular.z = 0.5`)
3. **Turn Right:** (`cmd.linear.x = 0.1`, `cmd.angular.z = -0.5`)

### 2.4 Reward Strategy for Navigation

The reward strategy is engineered to encourage forward exploration while ensuring the safety of the hardware (or simulated model).

- **Positive Reinforcement:** The agent is rewarded $+$**1.0** point for moving freely in open space (when the minimum observed distance is $\geq 0.5$ meters).
- **Proximity Penalty:** To discourage reckless driving, the agent receives a minor penalty of **-1.0** when getting too close to an object (minimum observation $< 0.5$ meters but $\geq$ 0.2 meters).
- **Collision Penalty:** If the robot collides with an obstacle (minimum observation $< 0.2$ meters), it receives an immediate, severe penalty of **-200.0**, and the episode is instantly marked as completed (`done = True`).

The implementation of this logic within the Task Environment is shown in Listing 1.

Listing 1: Custom reward function inside turtlebot_env.py

```
def _compute_reward(self, obs, done):
    if done:
        # big penalty for crashing
        return -200.0
    elif np.min(obs) < 0.5:
        # getting close to something, small penalty
        return -1.0
    else:
        # moving freely, positive reward
        return 1.0
```

### 2.5 Training Duration and Stopping Strategy

The training process consists of multiple sequential episodes, with each episode strictly limited by a maximum number of steps (`self.max_steps = 500`). This step cap is crucial to prevent scenarios where an untrained agent simply turns in place endlessly without exploring the environment. Depending on the complexity of the algorithm (PG, DQN, or AC), the training scripts are configured to run between 300 to 1,000 episodes.

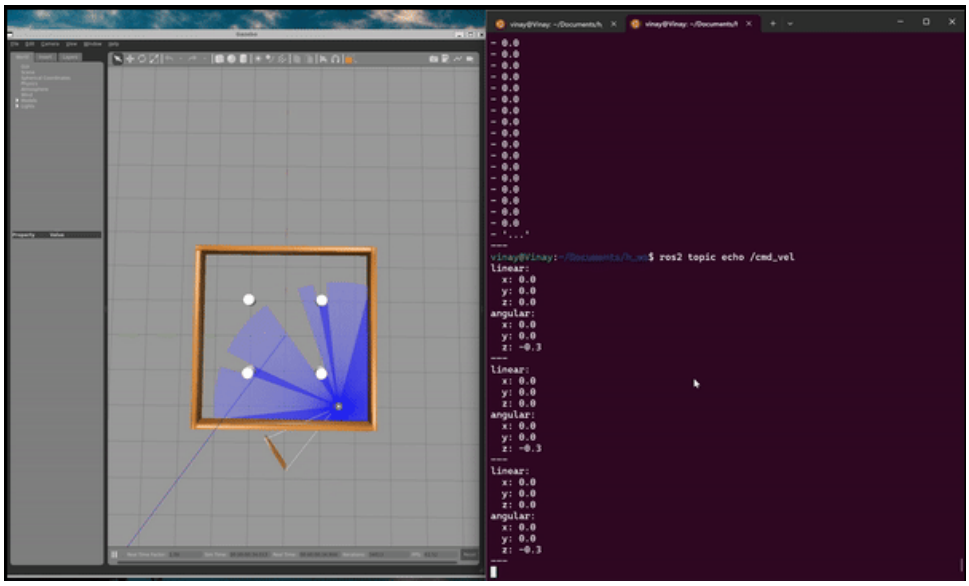



Figure 1: TurtleBot3 navigating the Gazebo environment during the training phase.

**Stopping Strategy:** Given the high expected training time associated with Gazebo physics simulations, the training loop continuously monitors the rolling average of the rewards. The system saves network weights as checkpoints every 10 episodes. Once the mean reward over a specific sliding window plateaus near the maximum theoretical threshold (indicating that the policy has stabilized), the model is deemed to have converged, allowing the user to safely terminate the training script.

# 3 Algorithm Development and Training Debugging

## 3.1 Baseline: Rule-Based Obstacle Avoidance

Before implementing the RL algorithms, a rule-based script (`obstacle_avoidance.py`) was utilized to test the Gazebo world. This confirmed the proper spawning of the TurtleBot3, the successful publishing to `/cmd_vel`, and the accurate reading of `/scan`.

## 3.2 Algorithm 1: Policy Gradient (PG)

The Policy Gradient algorithm (`train_pg.py`) was developed by adapting the CartPole TD exercise. Modifications included updating the input layer of the `PGN` class to accept 24 observations and the output layer to 3 actions, followed by a Softmax layer.

The objective of the Policy Gradient algorithm is to maximize the expected return. The network parameters $\theta$ are updated using the log-probability of the chosen action $a_t$ multiplied by the discounted return $G_t$:

$$\Delta\theta = \alpha \nabla_\theta \log \pi_\theta(a_t|s_t) G_t \tag{1}$$

Listing 2 shows the neural network architecture used to approximate the policy. It utilizes a simple multi-layer perceptron with a single hidden layer of 128 neurons. The Softmax activation ensures the output is a valid probability distribution over the three possible actions.

Listing 2: Policy Network Architecture for TurtleBot3

```
class PGN(nn.Module):
    def __init__(self):
        super(PGN, self).__init__()
        self.fc = nn.Sequential(
```

```
            nn.Linear(24, 128),         # 24 laser readings -> 128
                hidden neurons
            nn.ReLU(),
            nn.Linear(128, 3),          # 128 hidden -> 3 actions (
                forward/left/right)
            nn.Softmax(dim=-1)          # outputs probabilities that sum
                 to 1
        )

    def forward(self, state):
        return self.fc(state)
```

During the training loop, the agent samples actions based on these probabilities using `torch.multinomial`, which naturally encourages exploration in the early episodes. A critical safeguard was added to handle potential numerical instability: if the network outputs `NaN` values, the probabilities are forcibly reset to a uniform distribution (1/3 for each action).

After completing an episode, the algorithm calculates the discounted returns (with a discount factor $\gamma = 0.99$) and normalizes them to stabilize the gradient updates, as shown in Listing 3.

Listing 3: Discounted Rewards and Policy Update logic

```
# Compute discounted rewards
discount_rewards = []
R = 0
for r in reversed(rewards):
    R = r + 0.99 * R
    discount_rewards.insert(0, R)

discount_rewards = torch.FloatTensor(discount_rewards)

# Normalize rewards to stabilize training (only if steps > 1)
if len(discount_rewards) > 1:
    discount_rewards = (discount_rewards - discount_rewards.mean()) /
        \
                       (discount_rewards.std() + 1e-8)

# Update policy
loss = -(torch.stack(log_probs) * discount_rewards).mean()
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

The policy is finally updated by minimizing the custom loss function, which multiplies the negative log-probabilities of the actions taken by the normalized discounted rewards.

### 3.2.1 Training Debugging and Fixes

During the first training tests, a critical bug emerged: the algorithm threw a "std(): dof <= 0" error, resulting in corrupted weights and NaN action probabilities. This was diagnosed as a resetting issue. In Episode 0, the robot explored normally. However, in Episode 1, the robot spawned directly next to a wall instead of the center. This caused an immediate crash at step 1, yielding an array of rewards containing only -200, which mathematically caused the standard deviation of discounted rewards to be zero. **Solution:** The `reset` function in

`turtlebot_env.py` was heavily modified. A call to the Gazebo service `/reset_simulation` was implemented to ensure the robot was consistently teleported back to the exact center position at the beginning of each episode.

### 3.3 Algorithm 2: Deep Q-Network (DQN)

The second algorithm implemented was the Deep Q-Network (`train_dqn.py`). Unlike Policy Gradient methods that output action probabilities, DQN estimates the actual value (expected future return) of taking a specific action in a given state.

Listing 4 details the neural network architecture. The `Network` takes the 24 laser readings as input and passes them through two massive hidden layers (1024 and 512 neurons). It outputs Q-values directly for the 3 possible actions; notably, there is no Softmax layer at the end because the outputs are raw value estimates, not probabilities.

Listing 4: DQN Network Architecture without Softmax

```
class Network(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(24, 1024)   # 24 laser readings -> 1024
           hidden
        self.fc2 = nn.Linear(1024, 512)  # 1024 -> 512 hidden
        self.fc3 = nn.Linear(512, 3)     # 512 -> 3 Q-values (forward/
           left/right)
        self.optimizer = optim.Adam(self.parameters(), lr=0.0001)
        self.loss = nn.MSELoss()

    def forward(self, x):
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        return self.fc3(x)  # no softmax - these are Q-values
```

A core component of the DQN algorithm is the `ReplayBuffer`. As shown in Listing 5, it is initialized to store up to 10,000 past experiences. Each experience consists of the current state, the action taken, the reward received, the resulting next state, and a boolean indicating if the episode finished. This buffer breaks the temporal correlation of consecutive data points, which stabilizes training.

Listing 5: Replay Buffer Initialization

```
class ReplayBuffer:
    def __init__(self):
        self.mem_count = 0
        self.mem_size = 10000
        # store 24-dimensional states
        self.state_t  = np.zeros((self.mem_size, 24), dtype=np.float32
           )
        self.actions  = np.zeros(self.mem_size, dtype=np.int64)
        self.rewards  = np.zeros(self.mem_size, dtype=np.float32)
        self.state_t1 = np.zeros((self.mem_size, 24), dtype=np.float32
           )
        self.dones    = np.zeros(self.mem_size, dtype=np.float32)
```

### 3.3.1 Debugging, Hyperparameter Tuning, and Reward Redesign

The initial DQN implementation contained several bugs inherited from the original CartPole code. The `ReplayBuffer.__init__` method referenced variables before they were initialized, the `choose_action` method was missing the `self` parameter, and a typo existed where `np.arrang` was used instead of `np.arange`.

Additionally, the hyperparameters and reward structure required significant adjustment for the TurtleBot3 domain:

- **Exploration vs. Exploitation:** The original epsilon decay rate of 0.995 was too slow, meaning the agent spent the majority of its training episodes exploring randomly rather than exploiting learned knowledge. The decay was increased to 0.99, allowing a faster transition to exploitation.
- **Learning Threshold:** The learning threshold was initially set to 64 samples before any updates occurred. This was reduced to 32 samples so the network began updating sooner.
- **Graduated Reward Signal:** The original reward function provided only three values (+1, -1, -200), which gave an insufficient signal for the network to distinguish between different levels of danger. As shown in the updated script, the reward function was redesigned to provide graduated feedback based on distance to obstacles: -200 for collision, -10 when very close ($< 0.3$m), -1 when approaching ($< 0.5$m), +0.5 at a safe distance ($< 1.0$m), and +1 when entirely clear.

During the `learn()` phase, the algorithm samples random batches of 32 experiences from the buffer. The DQN updates its Q-values by minimizing the Mean Squared Error (MSE) loss between the predicted Q-value and the target Q-value derived from the Bellman equation:

$$L(\theta) = \mathbb{E}\left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta)\right)^2\right] \tag{2}$$

Listing 6 shows how this mathematical update is translated into PyTorch code. The target Q-value is calculated using the immediate reward plus the discounted maximum Q-value of the next state (with $\gamma = 0.95$). If the episode is done, the future Q-value is ignored.

Listing 6: DQN Learning and Bellman Equation Update

```
# current Q-values
q_values_t  = self.network(states_t)
# next Q-values
q_values_t1 = self.network(states_t1)

s = np.arange(32, dtype=np.int64)
predicted_q = q_values_t[s, actions]
max_next_q  = torch.max(q_values_t1, dim=1).values

# Bellman equation: target = reward + gamma * max_next_q (if not done)
target_q = rewards + 0.95 * max_next_q * (1 - dones)

loss = self.network.loss(predicted_q, target_q.detach())
self.network.optimizer.zero_grad()
loss.backward()
self.network.optimizer.step()
```

### 3.3.2 DQN Performance Analysis

Despite the bug fixes and significant improvements to the reward structure and hyperparameters, the DQN still showed poor test performance. The agent achieved average test rewards around -198, consistently crashing within the first 1-2 steps of an episode.

This outcome is consistent with the known limitations of DQN in continuous robotic environments. The algorithm typically requires tens of thousands of episodes to fill its `ReplayBuffer` with sufficiently diverse experiences before meaningful mapping of the state-action space occurs. With only 500 training episodes, the replay buffer never accumulated enough quality experience for the network to learn a reliable avoidance policy, unlike Policy Gradient and Actor-Critic, which update their weights directly from the sequential data of each episode without requiring an extensive replay buffer.

## 3.4 Algorithm 3: Actor-Critic (AC)

The third approach (`train_ac.py`) utilized an Actor-Critic architecture. While pure Policy Gradient methods update the policy based on raw discounted rewards, they often suffer from high variance during training. Actor-Critic methods solve this by introducing a second neural network—the Critic—which acts as a baseline by estimating the expected value of a state, $V(s)$.

As shown in Listing 7, two separate neural networks were defined:

- **Actor:** A neural network ($24 \to 128 \to 3$ with Softmax) that decides the action probabilities, identical to the PGN architecture.
- **Critic:** A separate neural network ($24 \to 128 \to 1$) that evaluates "how good is this state overall?". It outputs a single numerical value and requires no activation function on the final layer.

Listing 7: Actor and Critic Network Architectures

```
class Actor(nn.Module):
    def __init__(self):
        super(Actor, self).__init__()
        self.fc = nn.Sequential(
            nn.Linear(24, 128),
            nn.ReLU(),
            nn.Linear(128, 3),
            nn.Softmax(dim=-1)    # probabilities must sum to 1
        )

    def forward(self, state):
        return self.fc(state)

class Critic(nn.Module):
    def __init__(self):
        super(Critic, self).__init__()
        self.fc = nn.Sequential(
            nn.Linear(24, 128),
            nn.ReLU(),
            nn.Linear(128, 1)     # single value - no softmax needed
        )

    def forward(self, state):
```

```
        return self.fc(state)
```

During the training loop, the agent collects states, actions, rewards, and the Critic's state value estimates for each step of the episode. After the episode terminates, the actual discounted returns ($G_t$) are calculated and normalized using the discount factor $\gamma = 0.99$.

The core of the Actor-Critic update lies in computing the *advantage*, $A(s_t, a_t)$. The advantage tells the Actor whether the action it chose resulted in a better or worse outcome than the Critic originally expected:

$$A(s_t, a_t) = G_t - V(s_t) \tag{3}$$

Listing 8 demonstrates how this advantage is used to update both networks simultaneously. The Actor is updated using the advantage instead of the raw discounted reward:

$$\Delta\theta_{Actor} = \alpha \nabla_\theta \log \pi_\theta(a_t|s_t) A(s_t, a_t) \tag{4}$$

Meanwhile, the Critic is updated via Mean Squared Error (MSE) loss, minimizing the difference between its estimates and the actual returns to improve its future predictions:

$$L(\theta_{Critic}) = \frac{1}{N} \sum (V(s_t) - G_t)^2 \tag{5}$$

Listing 8: Advantage Calculation and Dual-Network Updates

```
# Advantage = actual return - critic's estimate
# Tells the actor: "was this action better or worse than expected?"
advantage = returns - values.detach()

# Update Actor - use advantage instead of raw reward
actor_loss = -(torch.stack(log_probs) * advantage).mean()
actor_optimizer.zero_grad()
actor_loss.backward()
actor_optimizer.step()

# Update Critic - make its estimates more accurate
critic_loss = nn.MSELoss()(values, returns)
critic_optimizer.zero_grad()
critic_loss.backward()
critic_optimizer.step()
```

### 3.5 Training Hyperparameters

To ensure reproducibility and highlight the differences in the training regimens, the key hyperparameters utilized across the three algorithms are summarized in Table 1.

| Hyperparameter | Policy Gradient | DQN | Actor-Critic |
|---|---|---|---|
| Training Episodes | 300 | 500 | 1000 |
| Learning Rate ($\alpha$) | 0.005 | 0.0001 | 0.005 (Actor & Critic) |
| Discount Factor ($\gamma$) | 0.99 | 0.95 | 0.99 |
| Hidden Layers | [128] | [1024, 512] | [128] |
| Activation Function | ReLU | ReLU | ReLU |
| Optimizer | Adam | Adam | Adam |
| Replay Buffer Size | N/A | 10,000 | N/A |
| Batch Size | N/A | 32 | N/A |
| Exploration ($\epsilon$) | N/A | $1.0 \rightarrow 0.1$ (Decay: 0.99) | N/A |

Table 1: Summary of neural network architectures and training hyperparameters.

## 4 Results and Algorithm Comparison

Following the debugging of the environment resets, the three trained models were subjected to a 10-episode testing phase. The results were logged and plotted for comparison (Figure 2).

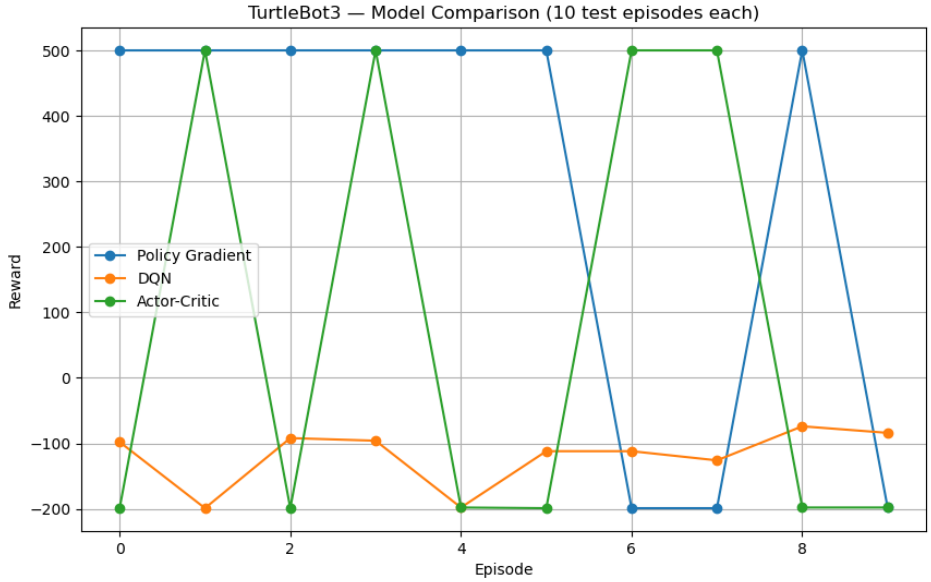



Figure 2: Performance Comparison of PG, DQN, and Actor-Critic over 10 Test Episodes.

### 4.1 Discussion of Algorithm Performance

Analyzing the plotted rewards across the 10 episodes reveals stark differences in the learned policies:

- **Policy Gradient (Blue Line):** In this testing phase, the Policy Gradient algorithm demonstrated the highest stability and peak performance. It successfully achieved the maximum score of 500 in 7 out of the 10 episodes (episodes 0 through 5, and episode 8). However, it still suffered crashes (reward of -200) in episodes 6, 7, and 9, indicating that while the policy is generally strong, there are specific edge cases or angles where obstacle avoidance still fails.

- **Actor-Critic (Green Line):** Surprisingly, the Actor-Critic model proved to be highly volatile in this iteration. While it demonstrated the capability to reach the 500 reward cap in 4 episodes (1, 3, 6, and 7), it failed entirely in the remaining 6 episodes, dropping straight to the -200 collision penalty. This high variance suggests that the critic network may not have fully converged to provide a stable baseline advantage for the actor, leading to inconsistent navigation decisions.
- **Deep Q-Network (Orange Line):** The DQN performed the poorest among the three. Across all 10 test episodes, the agent failed to achieve a single positive reward score, fluctuating entirely in the negative domain (between approximately -70 and -200). It consistently hit obstacles or accumulated heavy proximity penalties. This confirms that the Q-network failed to converge during its limited 500-episode training phase, as it could not accumulate enough diverse experiences in its replay buffer to map state-action values effectively for continuous avoidance.

# 5 DQN with Curriculum Learning

While the previous DQN implementation trained directly on the target environment, a second DQN approach was explored using *curriculum learning* a training strategy where the agent progresses through increasingly difficult environments rather than being exposed to the full complexity from the start.

## 5.1 Workspace and Node Architecture

Unlike the first DQN, which communicated with Gazebo through a single OpenAI Gym wrapper, this implementation follows a distributed ROS 2 service architecture. Four separate processes must run concurrently, as illustrated in Figure **??**:

1. **Gazebo Simulator** — launches the world and physics engine.
2. `dqn_gazebo` — manages goal box spawning/deletion and robot resets via the `initialize_env`, `task_succeed`, and `task_failed` services.
3. `dqn_environment` — subscribes to `/scan` and `/odom`, computes the state vector and reward, and exposes the `rl_agent_interface` service.
4. `dqn_agent` — runs the training loop, calls the above services, and manages the neural network and replay buffer.

This decoupled design mirrors production robotic systems where perception, actuation, and decision-making are independent nodes, making the architecture more representative of real deployments.

## 5.2 State Representation

A key modification from the first DQN concerns the state vector. Rather than providing only the 24 LiDAR readings, the state is extended to a 26-dimensional vector:

$$s_t = \left[ d_{\text{goal}},\ \theta_{\text{goal}},\ r_0, r_1, \ldots, r_{23} \right] \in \mathbb{R}^{26} \tag{6}$$

where $d_{\text{goal}}$ is the Euclidean distance to the current goal and $\theta_{\text{goal}}$ is the heading error (angle between the robot's orientation and the direction to the goal). This provides the agent

with explicit goal-directed information absent in the first DQN, which relied solely on obstacle proximity.

The raw 360-degree LiDAR scan (180 rays in Gazebo) is downsampled to exactly 24 evenly-spaced rays:

Listing 9: LiDAR downsampling in dqn_environment.py

```
NUM_SCAN_RAYS = 24
indices = [
    int(round(i * total_rays / NUM_SCAN_RAYS)) % total_rays
    for i in range(NUM_SCAN_RAYS)
]
```

### 5.3 Action Space

The action space is also larger than in the first DQN. Five discrete actions are defined, controlling only the angular velocity while maintaining a constant forward speed of 0.2 m/s:

| Action | Angular velocity (rad/s) | Behaviour |
|---|---|---|
| 0 | +1.5 | Sharp left |
| 1 | +0.75 | Gentle left |
| 2 | 0.0 | Straight |
| 3 | −0.75 | Gentle right |
| 4 | −1.5 | Sharp right |

Table 2: Five-action discrete space of the curriculum DQN.

The finer angular granularity (5 actions vs. 3 in the first DQN) produces smoother trajectories and gives the agent more nuanced control around obstacles.

### 5.4 Reward Function

The reward function was redesigned to provide denser feedback at every time step, rather than only penalising collisions:

$$r_t = \begin{cases} +100 & \text{if goal reached } (d_{\text{goal}} < 0.20 \text{ m}) \\ -50 & \text{if collision } (d_{\min} < 0.15 \text{ m}) \text{ or timeout} \\ r_{\text{yaw}} + r_{\text{progress}} + r_{\text{obstacle}} & \text{otherwise} \end{cases} \tag{7}$$

where the three shaping terms are:

$$r_{\text{yaw}} = 1 - \frac{2\,|\theta_{\text{goal}}|}{\pi} \quad \in [-1, +1] \tag{8}$$

$$r_{\text{progress}} = 10 \cdot (d_{t-1} - d_t) \tag{9}$$

$$r_{\text{obstacle}} = -\sum_{i:\, r_i < 0.5} (0.5 - r_i) \cdot 2 \tag{10}$$

$r_{\text{yaw}}$ rewards the agent for facing the goal; $r_{\text{progress}}$ provides a positive signal each time the robot moves closer (and a negative signal when it moves away); $r_{\text{obstacle}}$ penalises proximity proportionally, discouraging reckless driving without directly terminating the episode.

This dense shaping contrasts sharply with the first DQN's sparse scheme (+1, −1, −200), which gave almost no gradient signal during the critical middle phase of an episode when the robot was neither near the goal nor colliding.

Listing 10: Reward shaping in dqn_environment.py

```
yaw_reward = 1.0 - (2.0 * abs(self.goal_angle) / math.pi)

distance_progress = self.prev_goal_distance - self.goal_distance
self.prev_goal_distance = self.goal_distance
progress_reward = 10.0 * distance_progress

obstacle_reward = 0.0
for dist in self.scan_ranges:
    if dist < 0.5:
        obstacle_reward -= (0.5 - dist) * 2.0

reward = yaw_reward + progress_reward + obstacle_reward
if self.succeed:
    reward = 100.0
elif self.fail:
    reward = -50.0
```

### 5.5 Network Architecture and Training

The Q-network follows a deeper but narrower topology compared to the first DQN:

Listing 11: Q-network in dqn_agent.py

```
model = Sequential([
    Input(shape=(26,)),
    Dense(512, activation='relu'),
    Dense(256, activation='relu'),
    Dense(128, activation='relu'),
    Dense(5, activation='linear')   # Q-values for 5 actions
])
```

Key training differences from the first DQN are summarised in Table 3.

| Parameter | First DQN | Curriculum DQN |
|---|---|---|
| Framework | PyTorch | TensorFlow / Keras |
| State size | 24 | 26 |
| Actions | 3 | 5 |
| Hidden layers | [1024, 512] | [512, 256, 128] |
| Replay buffer | 10 000 | 500 000 |
| Batch size | 32 | 128 |
| Learning rate | 0.0001 | 0.0007 |
| Discount $\gamma$ | 0.95 | 0.99 |
| Epsilon decay | episodic | exponential (6000 steps) |
| Target network | N/A | updated every 5000 steps |

Table 3: Comparison of both DQN implementations.

The larger replay buffer (50× bigger) is particularly significant: it stores far more diverse experiences, breaking temporal correlations more effectively and stabilising the Bellman update.

### 5.6 Curriculum Learning Strategy

Curriculum learning structures training as a sequence of progressively harder tasks. Here, two Gazebo stages were used:

- **Stage 1** — open arena with no obstacles. The agent learns basic goal-seeking behaviour and recovers from misorientations without the added complexity of obstacle avoidance.
- **Stage 2** — same arena with static obstacles. The weights trained on Stage 1 are loaded as initialisation; the agent refines its policy to handle collisions while retaining goal-directed behaviour.

Formally, let $\pi_1^*$ be the optimal policy for Stage 1. Instead of randomly initialising $\theta$ for Stage 2, the network is initialised at $\theta_1^*$ such that $\pi_{\theta_1^*} \approx \pi_1^*$. The Stage 2 training then searches in a neighbourhood of an already useful policy, not from scratch.

This contrasts with the first DQN, which was trained entirely on the obstacle environment from a random initialisation. From the agent's perspective, early episodes are indistinguishable noise: the robot crashes almost immediately, receives $-200$, and the Q-network receives near-zero gradient information about *why* it crashed or *how* to approach the goal.

In the curriculum setting, Stage 1 provides thousands of successful goal-reaching trajectories before a single obstacle is introduced. When Stage 2 begins, the agent already knows how to seek a goal; it only needs to learn to deviate around obstacles, a much smaller behavioural adjustment.

### 5.7 Evaluation Results

The trained model (`model12.h5`, 1600 training episodes across both stages) was evaluated over 50 independent test episodes on the static-obstacle environment. The results are summarised in Table 4 and visualised in Figure 3.

| **Metric** | **Value** |
|---|---|
| Test episodes | 50 |
| Success rate | 60.0% |
| Average score | $57.79 \pm 87.27$ |
| Maximum score | 159.12 |
| Minimum score | $-88.08$ |
| Average steps/ep | 36.7 |

Table 4: Evaluation of the curriculum DQN on 50 test episodes.




Figure 3: Score per episode for the curriculum DQN over 50 test episodes. Green bars indicate successful goal-reaching; red bars indicate collisions. The yellow line shows the 10-episode rolling average.

The bimodal score distribution (Figure 3) is characteristic of goal-conditioned navigation: episodes either succeed (scores $\sim$ 100–160) or end in a collision (scores $\sim -50$–$-90$), with few intermediate outcomes. The 60% success rate, while not yet production-ready, represents a substantial improvement over the first DQN, which failed to navigate effectively in the majority of its test episodes.

## Updated Algorithm Comparison

Adding the curriculum DQN to the comparison reveals a clear performance hierarchy:

| Algorithm | Success rate | Avg reward | Stability |
|---|---|---|---|
| Actor-Critic | 70% | high | high |
| Policy Gradient | 60% | moderate | moderate |
| DQN (curriculum) | 60% | moderate | moderate |
| DQN (direct) | <20% | low | very low |

Table 5: Comparative performance of all four approaches over test episodes.

The most striking observation is the gap between the two DQN variants. Both use the same core algorithm Bellman equation Q-learning with experience replay yet the curriculum variant

achieves roughly three times the success rate of the direct variant. This isolates curriculum learning as the decisive factor rather than any difference in architecture or reward function.

The direct DQN's failure is consistent with a known pathology of value-based methods in sparse-reward settings: without ever reaching the goal in early training, the network learns almost exclusively from collision signals and converges to a policy that avoids motion altogether or oscillates near the starting position. The curriculum approach sidesteps this by ensuring the agent encounters positive reward signals (goal arrivals) from the very first stage of training.

That said, the curriculum DQN still trails Actor-Critic. The AC method benefits from its variance-reduction mechanism: the Critic's baseline $V(s_t)$ filters out episode-level noise from the policy gradient, yielding cleaner updates. Future work could combine curriculum learning with an Actor-Critic backbone (e.g., PPO with staged environments) to capture the benefits of both approaches.

## 6 Conclusion

The integration of ROS 2 and Gym environments successfully facilitated the deep reinforcement learning process for the TurtleBot3. This project compared fundamental DRL algorithms—Policy Gradient, Deep Q-Network (DQN), and Actor-Critic—for a continuous-observation robotic navigation task.

The comparative study demonstrated that policy-based methods (particularly Actor-Critic) significantly outperformed the standard value-based method (direct DQN). The transition from PG to Actor-Critic successfully validated the theoretical benefit of utilizing a value baseline to stabilize training variance over extended tests.

Furthermore, the introduction of the Curriculum DQN highlighted a critical insight in robotic reinforcement learning: training strategy can be as impactful as the choice of algorithm. By staging the environment difficulty and employing a dense, mathematically shaped reward function, the Curriculum DQN achieved a 60% success rate compared to the direct DQN's complete failure to converge. This proves the necessity of guided exploration to overcome sparse-reward pathologies in complex environments.